

Computing Methods for Physics – 21 November 2022

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 12:00 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Implement a C++ abstract class **Solid** to represent rotating solids with uniform density and a standard shape. Provide it with attributes to represent its mass M , a single characteristic length scale (this will suffice for, e.g., a thin rod, a sphere, a square, etc., but not for, e.g., a cylinder, a rectangle, etc.), its angular rotation frequency ω , its moment of inertia I , its kinetic energy K , and the distance d between the rotation axis and an axis parallel to it that goes through the center of mass (CoM).

I cannot be calculated until the specific shape and mass of the solid is known. The kinetic energy

$$K = \frac{1}{2}I\omega^2$$

cannot be calculated without knowing I and how the solid spins (i.e., ω and d ; remember that $I = I_{\text{CoM}} + Md^2$).

Also implement three concrete derived classes: **Sphere** (to represent a rotating solid sphere), **Rod** (to represent a thin solid rod that rotates along an axis perpendicular to it), and **Rectangle** (to represent a thin rectangular slate that rotates along an axis perpendicular to the plane that contains it). The moments of inertia for these solids, with respect to a rotation axis going through their CoMs, are:

$$\begin{aligned} I_{\text{Sphere}} &= \frac{2}{5}MR^2 \\ I_{\text{Rod}} &= \frac{1}{12}ML^2 \\ I_{\text{Rectangle}} &= \frac{1}{12}M(L^2 + W^2) \end{aligned}$$

Notice that handling the rectangle requires an additional length scale (the width, W).

Provide each class with getters, setters, a **Print** method, and with a method **Spin** to set the object in rotation (d and ω are the inputs of this method).

Include a file `app.cpp` that showcases the classes you implemented.

Part 2 – Python

Write a Python module `int_tools.py` that provides the user with methods capable of addressing the following tasks.

1. Calculating the binomial coefficient of n and k — This is a natural number defined as

$$\binom{n}{k} = \begin{cases} \frac{n!}{k!(n-k)!} & \text{if } k \in [1, n] \\ 1 & \text{if } k = 0 \end{cases}. \quad (1)$$

2. Printing the first M lines of “Pascal’s triangle” — The n -th line of this triangle is given by $\binom{n}{0}, \binom{n}{1}, \dots, \binom{n}{n}$.
3. Calculating the probability that an unbiased coin, tossed n times, will come up heads k times — This is given by $P(n, k) = 2^{-n} \binom{n}{k}$.
4. Finding all the prime numbers up to N — This must be done as follows. Initialize a collection with the first prime number, i.e., 2. Then for $3 \leq n \leq N$, check whether n is divisible by any of the primes in the collection up to and including $\sqrt{n}$. As soon as this condition is met, stop checking it (because at this point you know n is not prime); if the condition is never met, update the collection of prime numbers with n .
5. Provide the ability to store the results of using the methods for point 2 or 4 to a binary file.

Use your module to

- determine the prime numbers up to $N = 10^4$, and hand in your output as a binary file;
- calculate the probability that 100 coin tosses return 60 heads or more;
- create a plot displaying how many prime numbers are contained in a line of “Pascal’s triangle” as a function of the line number. [Limit the plot to lines with entries $\leq N = 10^4$ and submit your plot as a `.png`.]

Suggestion: if your code is slow, work with smaller values of N , and of course try to optimize it.